

Overview of Transformer architecture deep learning

Subject: Transformer architecture deep learning

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1.

Tokenization As the transformer architecture natively consists of operations over numbers (matrix multiplications, dot products, activation functions) rather than over text, there must first be a mapping from any input text to some numerical representation. This happens in three steps. First, the input text is treated by a preprocessor, which performs both textual transformations and splits the text into coarse-grained segments called pretokens. The latter is referred to as pretokenization. Second, each pretoken is segmented further into tokens by a tokenizer that expects to only see pretokens output by its preprocessor. Each token it produces is a string of one or more characters belonging to a finite set of strings called the vocabulary V . Third, because the vocabulary is finite and known beforehand, each token can be assigned an integer identifier, and this mapping is applied to the sequence of tokens to represent any input text as a numerical sequence. Since this mapping is bijective, the output side can produce a sequence of integer identifiers which can then be turned back into tokens. After undoing some of the preprocessing, the result is again legible text. Training a tokenizer (sometimes referred to as vocabularization) means finding a suitable vocabulary V , but also learning how to use it, since any given string s of length $|s|$ has $2^{|s|-1}$ hypothetical segmentations, some of which containing segments that are not in the vocabulary. The most important hyperparameter during vocabularization is the vocabulary size $|V|$: when it is small, the learned vocabulary generally consists of characters and smaller strings, and words will be segmented into many tokens. At larger sizes, it becomes affordable to dedicate tokens to full words, although depending on the preprocessor and tokenizer, it is not necessarily the case that large vocabularies will always use the largest token(s) available to segment a word. Because tokens are not always full words, they may also be referred to as subwords and tokenization algorithms may be referred to as subword tokenizers. This is also to differentiate these systems from traditional terminology used in older information retrieval and natural language processing systems, where "tokenization" was used to denote what is today called "pretokenization" (very crudely: splitting into words). In tokenizers that produce tokens that are not part of the vocabulary, a special token that does belong to the vocabulary is used as a generic stand-in, written as "[UNK]" for "unknown". In principle, any string could be hidden by such an [UNK]. Indeed, in information retrieval, pretokenizers were themselves used as tokenizers (and also called "tokenizers") with a word-level vocabulary that contained an [UNK]. Commonly used subword tokenization algorithms are byte pair encoding (BPE) and the unigram language model (ULM), which each include a vocabularization algorithm and a dedicated segmentation algorithm. There also exist several segmentation algorithms that require no learning and can be applied given a vocabulary (produced by BPE or ULM, for example), like greedily recognising tokens in a pretoken by moving through it left-to-right. Well-known software implementations of subword tokenizers are Hugging Face's tokenizers Python package implemented in Rust, and the sentencepiece Python package

implemented in C++. The latter package is named as such because one of its configuration options allows disabling the built-in pretokenizer, hence effectively making entire sentences a pretoken and thus having the tokenizer see entire sentences, rather than individual words.

2.

```
/* second sublayer */ z_d_copy ← copy(z_d) for each t in 1:length(z_d) do z_d[t] ←
layer.layer_norm(z_d[t]) z_d ← layer.multihead_attention(z_d, z_e, z_e) for each t in 1:length(z_d) do
z_d[t] ← z_d[t] + z_d_copy[t]
```

3.

Each integer token identifier is converted into an embedding vector via a lookup table. Equivalently stated, it multiplies a one-hot representation of the token identifier by an embedding matrix M $\{\displaystyle M\}$. For example, if the input token's identifier is 3 $\{\displaystyle 3\}$, then the one-hot representation is $[0, 0, 0, 1, 0, 0, \dots]$ $\{\displaystyle [0,0,0,1,0,0,\dots]\}$, and its embedding vector is $\text{Embed}(3) = [0, 0, 0, 1, 0, 0, \dots] M$ $\{\displaystyle \mathrm{Embed}(3)=[0,0,0,1,0,0,\dots]M\}$. The token embedding vectors are added to their respective positional encoding vectors (see below), producing the sequence of input vectors. The dimension of an embedding vector is called hidden size or embedding size and written as d_{emb} $\{\displaystyle d_{\text{emb}}\}$. This size is written as d_{model} $\{\displaystyle d_{\text{model}}\}$ in the original transformer paper.

4.

$f(t) = (e^{it/r^k})_{k=0,1,\dots,d/2-1}$ $\{\displaystyle f(t)=\left(e^{it/r^k}\right)_{k=0,1,\dots,\frac{d}{2}-1}\}$ where $r = N^{2/d}$ $\{\displaystyle r=N^{2/d}\}$. The main reason for using this positional encoding function is that using it, shifts are linear transformations: $f(t + \Delta t) = \text{diag}(f(\Delta t))f(t)$ $\{\displaystyle f(t+\Delta t)=\mathrm{diag}(f(\Delta t))f(t)\}$ where $\Delta t \in \mathbb{R}$ $\{\displaystyle \Delta t \in \mathbb{R}\}$ is the distance one wishes to shift. This allows the transformer to take any encoded position, and find the encoding of the position n -steps-ahead or n -steps-behind, by a matrix multiplication. By taking a linear sum, any convolution can also be implemented as linear transformations: $\sum_j c_j f(t + \Delta t_j) = (\sum_j c_j \text{diag}(f(\Delta t_j)))f(t)$ $\{\displaystyle \sum_j c_j f(t+\Delta t_j)=\left(\sum_j c_j \mathrm{diag}(f(\Delta t_j))\right)f(t)\}$ for any constants c_j $\{\displaystyle c_j\}$. This allows the transformer to take any encoded position and find a linear sum of the encoded locations of its neighbors. This sum of encoded positions, when fed into the attention mechanism, would create attention weights on its neighbors, much like what happens in a convolutional neural network language model. In the author's words, "we hypothesized it would allow the model to easily learn to attend by relative position." In typical implementations, all operations are done over the real numbers, not the complex numbers, but since complex multiplication can be implemented as real 2-by-2 matrix multiplication, this is a mere notational difference.